

Newton School of Technology

7

How Software Works

Team Newton

C07: System & AI Essentials

Newton School of Technology

Join the lecture through the online portal

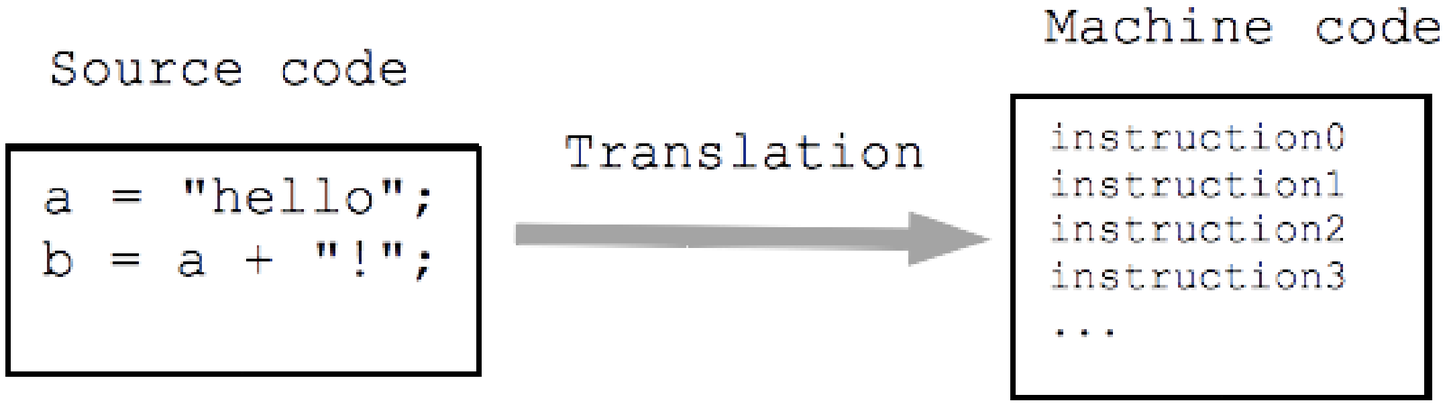
Recap: How does python run?

Python is a program that lives on your disk, usually at **/usr/bin/python3**.



So every time you run a **Python** script, two programs are involved. The Python program, and your script.

What the CPU actually understands?



- The CPU only understands **machine code**.
- Not Python. Not JavaScript. Specific patterns the chips recognise: add these, move this there, jump to that address.
- Every program you have ever run ends up as binary on a CPU. **No exceptions**.
- So the question is never whether translation happens. It always happens. The only question is when, and who does it.

Let’s take a step back

- Translation is not a fact of nature.
- Somebody had to invent the idea, and when she did, the entire industry told her she was wrong.
- She asked: “What if you wrote something closer to English, and a program translated it into machine code for you?”



Newton School of Technology

Hopper in David Letterman show!

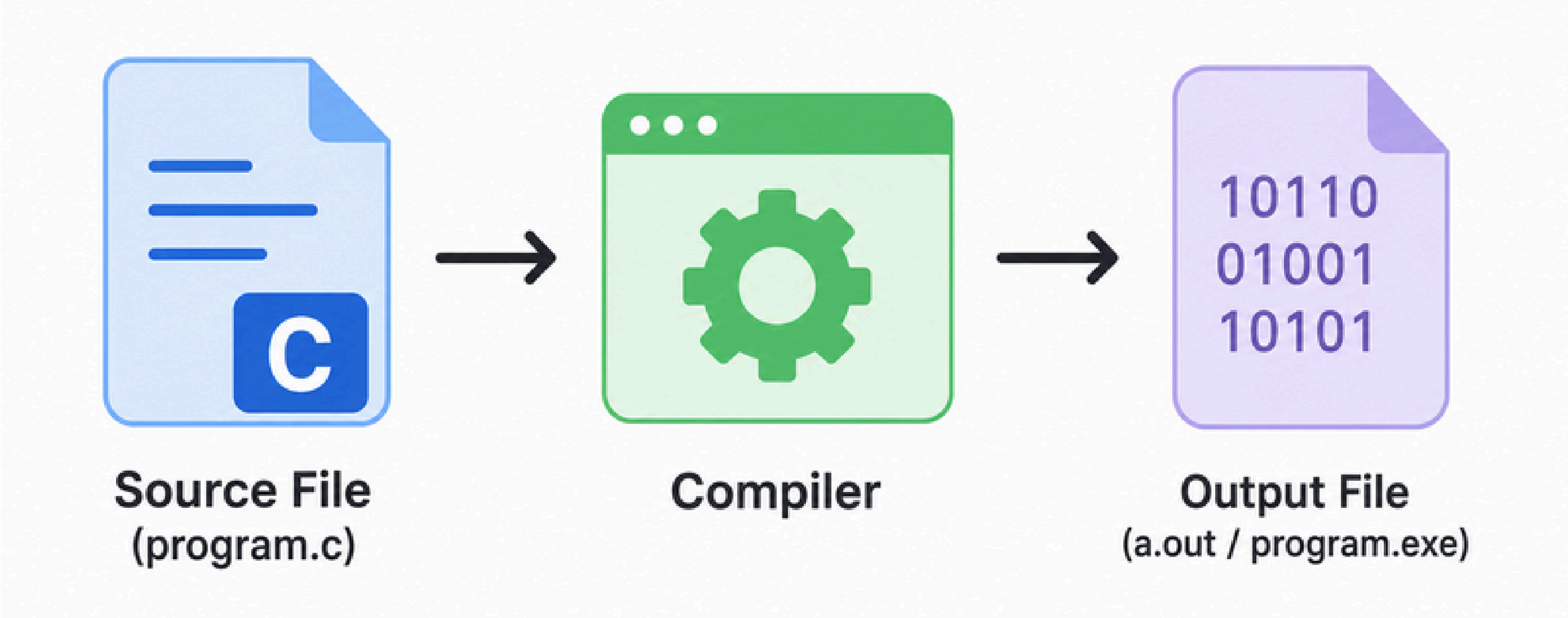
She's 79 here, just retired as a rear admiral. Watch what she pulls out of her bag

Newton School of Technology

The Compiler Path

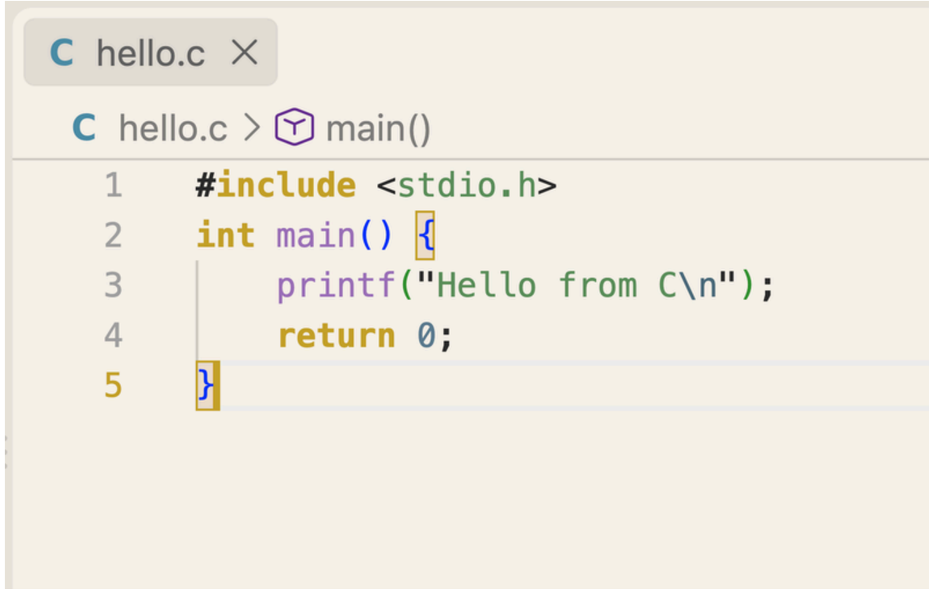
The Compiled program

Compilation is the process of translating an entire source code file into machine code before the program is executed.



The Compiled path

Newton School of Technology



- You write source code
- Compile it using an exisiting program
- Run the binary

```
› clang hello.c -o hello
› ls -la hello.c hello
-rwxr-xr-x  1 tanmay  staff  33432  6 Sep 21:53 hello
-rw-r--r--  1 tanmay  staff     77  6 Sep 21:52 hello.c
› ./hello
Hello from C
```

The Compiled path



Source File

1

The Compiled path



Source File

1



Compiler translates source file

2

The Compiled path



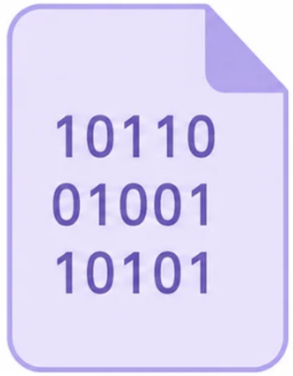
Source File

1



Compiler translates source file

2



Output File

3

The Compiled path



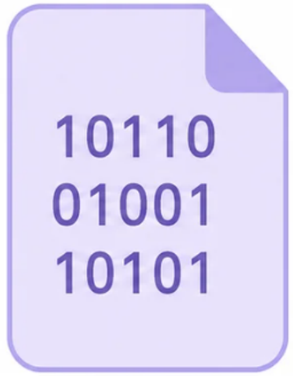
Source File

1



Compiler translates source file

2



Output File

3



CPU executes it directly

4

How to read the binary file?

Newton School of Technology

```
› cat hello
����
     @ H__PAGEZERO@__TEXT@@__text__TEXT`4`@__stubs__TEXT@
                                                        @
                                                          __cstring__TEXT@@__unw
ind_info__TEXT@X@@__DATA_CONST@@@@__got__DATA_CONST@H__LINKEDIT@@@@4@@`3@`@0@@È(

PÄ
   /usr/lib/dyld@|=d@@;@@.x@:@2 @*(@`
                                    8L/usr/lib/libSystem.B.dylib&@)@@@@@@@@{@@C
@R@
   ##########
```

This file has parts that aren’t human readable!

How to read the binary file?

```
❯ xxd hello | head -20
00000000: cffa edfe 0c00 0001 0000 0000 0200 0000  ................
00000010: 1100 0000 2004 0000 8500 2000 0000 0000  .... ..... .....
00000020: 1900 0000 4800 0000 5f5f 5041 4745 5a45  ....H..__PAGEZE
00000030: 524f 0000 0000 0000 0000 0000 0000 0000  RO..............
00000040: 0000 0000 0100 0000 0000 0000 0000 0000  ................
00000050: 0000 0000 0000 0000 0000 0000 0000 0000  ................
00000060: 0000 0000 0000 0000 1900 0000 8801 0000  ................
00000070: 5f5f 5445 5854 0000 0000 0000 0000 0000  __TEXT..........
00000080: 0000 0000 0100 0000 0040 0000 0000 0000  .........@......
00000090: 0000 0000 0000 0000 0040 0000 0000 0000  .........@......
000000a0: 0500 0000 0500 0000 0400 0000 0000 0000  ................
```

Displays file without unrecognized symbols

Print first 20 lines

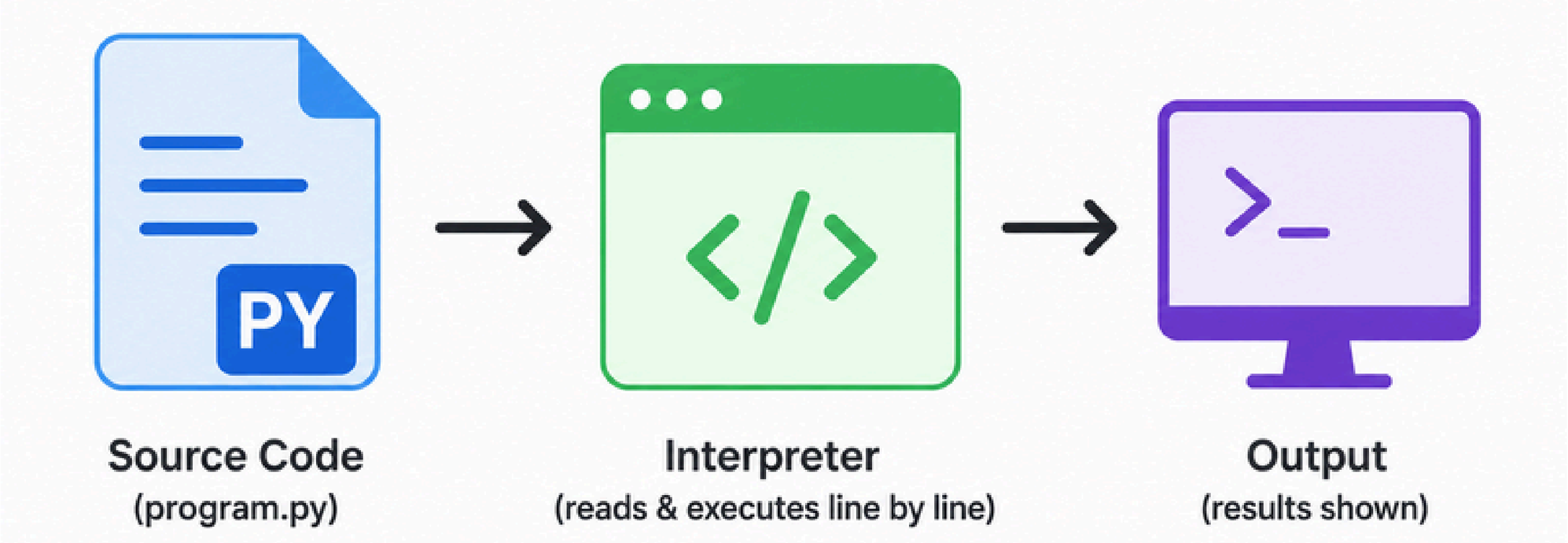


Newton School of Technology

The other way: Interpreted Path

The Interpreted Path

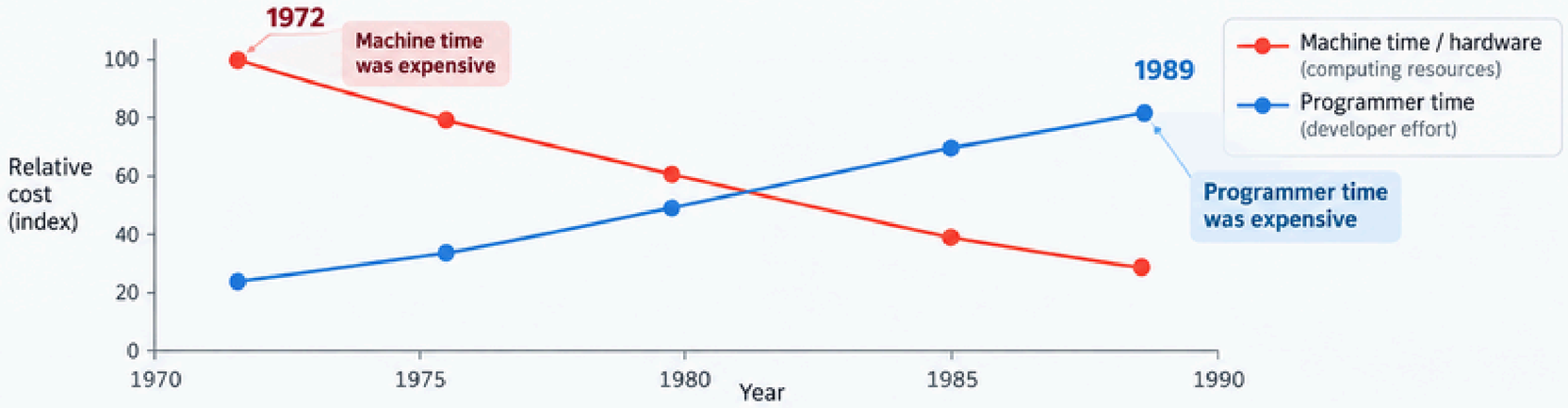
Interpretation is the process of translating and executing source code into machine code one line (or statement) at a time as the program runs.



Newton School of Technology

What became expensive over time?

The economic shift behind different programming language priorities



1972 — C



- Computer time was the expensive thing.
- Programmers were comparatively cheap.
- So it made sense to give programmers low-level control and efficient execution.
- That is the origin of the C language (1972).

Optimize for machine time.

1989 — Python



- Hardware was getting cheaper every year.
- Programmers were not.
- So it made sense to optimize for developer productivity, readability, and reduced complexity.
- That is the origin of the Python language (1989, first release in 1991).

Optimize for programmer time.

As hardware got cheaper and more powerful, the focus of programming languages shifted from machine efficiency to developer productivity.

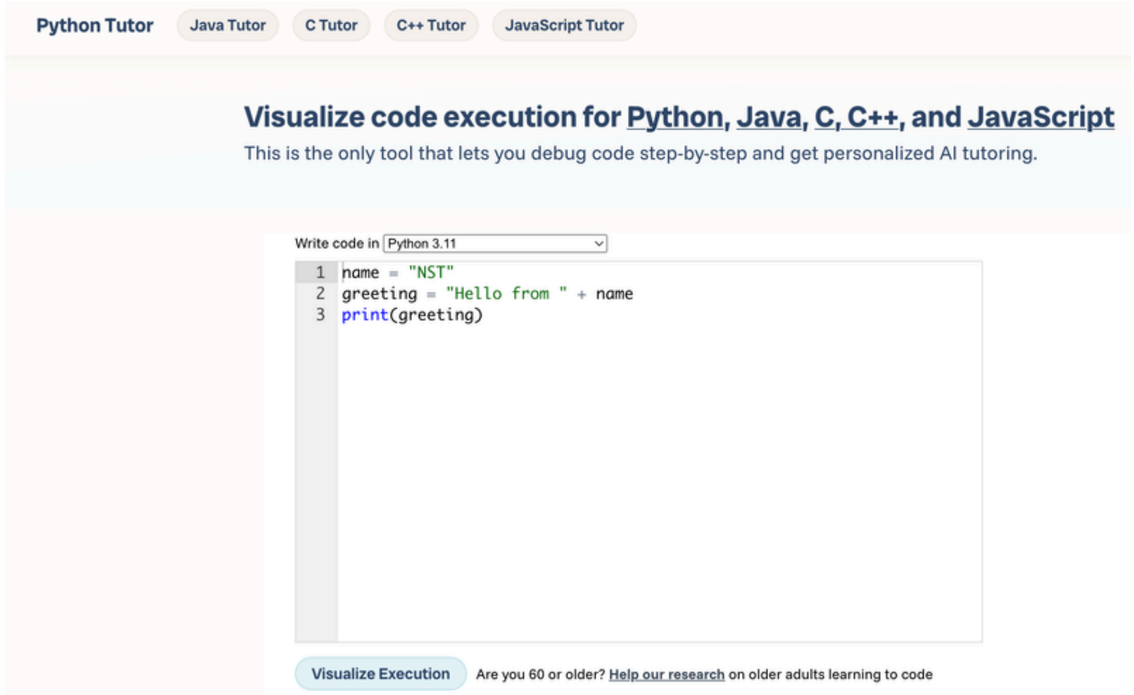
Why Interpreted languages?

- In 1972 optimised for machine time, because in 1972 computer time was the expensive thing and programmers were comparatively cheap. That is origin of C language
- By 1989 that had flipped. Hardware was getting cheaper every year. Programmers were not. That is origin of Python language.



python3 hello.py — Step by Step

Newton School of Technology





The Interpreted Path



Source File

1

The Interpreted Path



Source File

1



Interpreter reads
line 1 and translates it

2

The Interpreted Path



Source File

1



Interpreter reads
line 1 and translates it

2



CPU executes it
immediately

3

The Interpreted Path



Source File

1



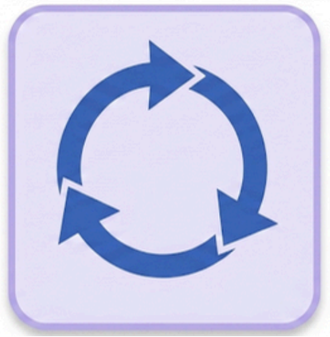
Interpreter reads
line 1 and translates it

2



CPU executes it
immediately

3



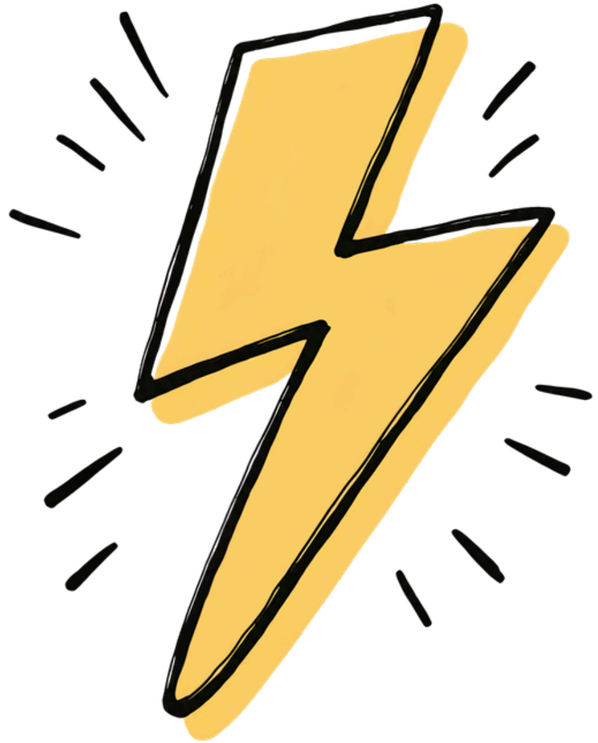
Repeat for line 2, 3 ...

4

Recap of what we saw today

Hopper’s wires

- 30 cm = 1 nanosecond
- How far electricity travels in a billionth of a second
- Set by the speed of light
- Not an engineering problem, just a constraint



Time taken to run a similar program

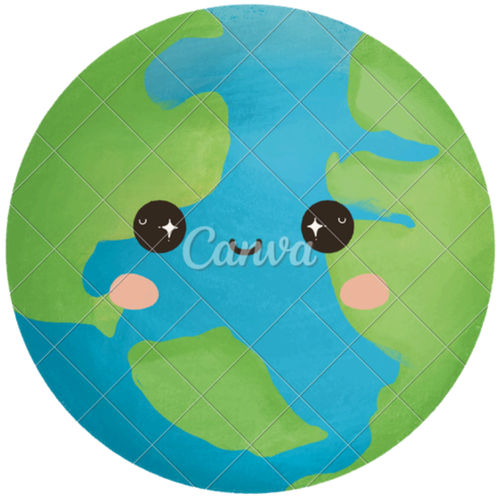
C	0.001 s
Python	0.1 s

Difference: 100,000,000 nanoseconds

100 million of Hopper's wires

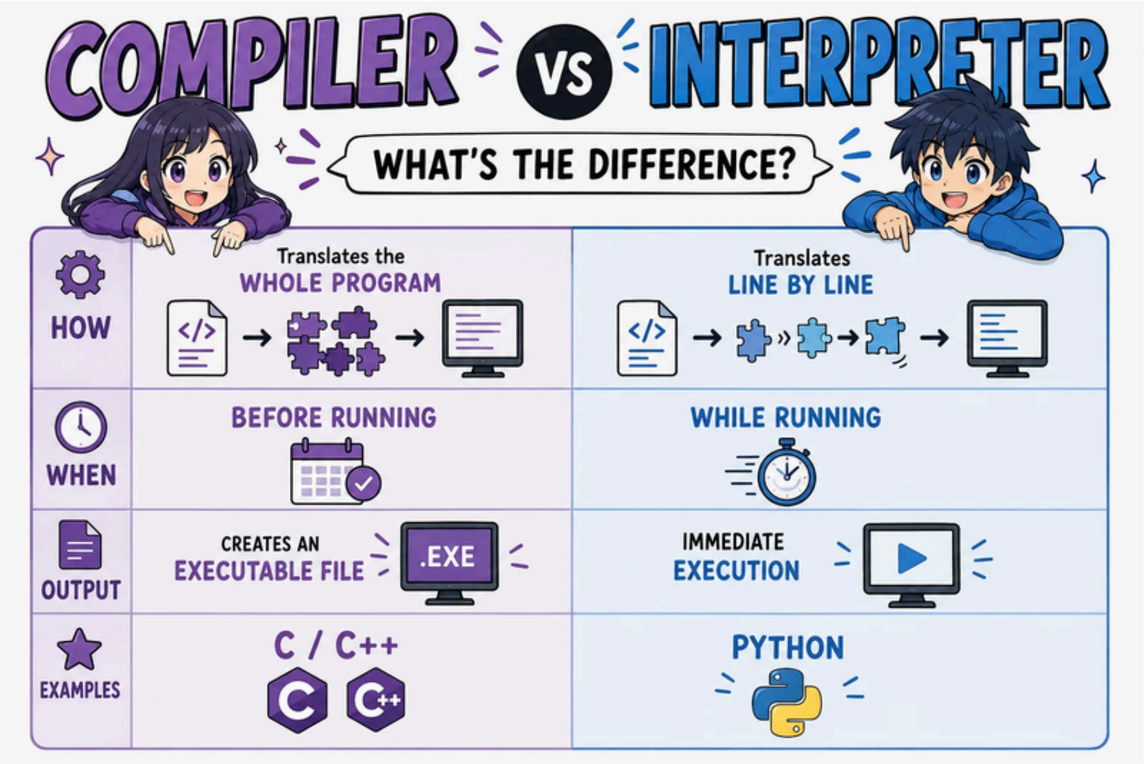
about 30,000 km

¾ of the way around the Earth



Where the time goes?

- Interpreter: translates while your program runs. Every line. Every run. Every time.
- Compiler: translated once. Last Tuesday. Never again.



https://lavenderbyte.com/blogs/news/compiler-vs-interpreter

Neither approach is wrong.
Different answers to one question: what's expensive?

- *1972: machine time is scarce → Ritchie builds C*
- *1989: human time is scarce → Guido builds Python*

Newton School of Technology

Quiz Time

Feedback

Now it's time to give your valuable feedback



Further Reading

- **Compiler Explorer**: [https://godbolt.org/]
- **Python Tutor**: [https://pythontutor.com/]
- "**Reflections on Trusting Trust:** [Reflections on Trusting Trust]
- **CPython source code on GitHub:** [https://github.com/python/cpython]
- **V8 blog**: [https://v8.dev/]

Newton School of Technology

Thank You!

Newton School of Technology

Deep Dive

Newton School of Technology

The Bootstrap Problem

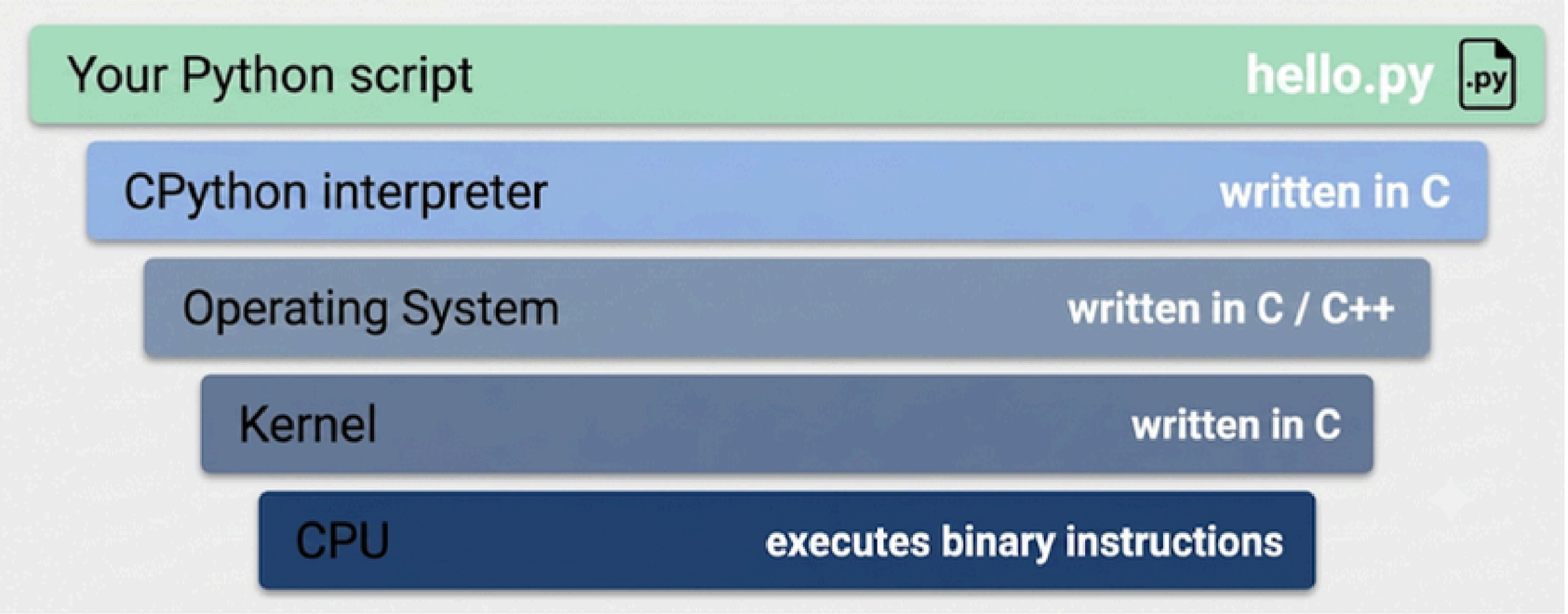
The Bootstrap problem

Newton School of Technology

1. Python **cannot** interpret itself before it exists. Something has to be already running on the machine, something that **compiles** to machine code directly with no interpreter in the loop, and **C** does that.
2. The Python interpreter is called CPython - the C is in the name.
3. This is the **bootstrap problem**: to build a tool that runs a **language,** you need a language that already runs.

The Software Stack

Newton School of Technology



The entire software stack has **C** at the **bottom**. Not because C is perfect, but because it solved the right problem at the right time and everything was **built** on top of it.

Why C – The Language of Systems ?

Newton School of Technology

BEFORE C

Assembly only

Systems were written directly in assembly — fast, but tied to one CPU architecture.

EARLY 1970S

C is created

Dennis Ritchie, Bell Labs, writes C to rewrite Unix — close to hardware, but portable across chips.

AFTER

Everything builds on it

Linux, macOS's Darwin layer, Windows — all written in C or C++. So are their compilers and interpreters.

Just-in-Time Compilation

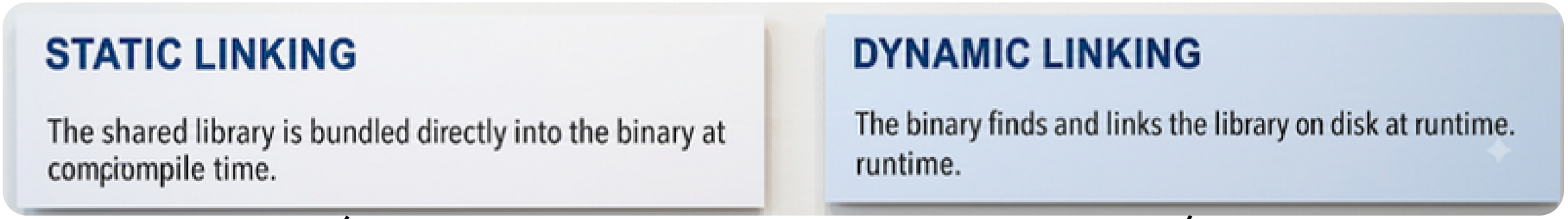
Newton School of Technology

JavaScript's **V8 engine** (Chrome and Node.js) starts by interpreting JavaScript, but compiles frequently run code to native binary on the fly.
This is why **JavaScript** is fast enough to power complex web applications despite being interpreted — the compiler runs while the program is already running.

Read further: “How V8 works” on the V8 blog (v8.dev)

Static vs Dynamic Linking

Newton School of Technology



Larger binary, self-contained, nothing to break later.

Smaller binary, used by most programs on your machine, which is why deleting a shared library can break several programs at once.

CPython & PyPy

Newton School of Technology

CPython is the reference Python interpreter, written in C. **PyPy** is an alternative interpreter written in Python itself using JIT compilation to outrun CPython in many cases.

The existence of multiple **implementations** of the same language shows that the language and the interpreter are separate things. Python is a specification, and CPython and PyPy are just two ways to run it.

Read further: **pypy.org** for the PyPy project